NetSage AI

Streamlit: https://cisco-netstage-cymhgnzo4p2hct4g2mwhhm.streamlit.app/

Demo Video: https://youtu.be/UR_gMDkmrrY

NetSage AI is a Streamlit-based network troubleshooting dashboard for Cisco-style lab networks. It helps identify likely network faults from a selected lab case, shows the evidence behind the diagnosis, and suggests commands or fix steps.

The project follows a human-in-the-loop approach: the system does not automatically change any router or switch configuration. A human operator reviews the diagnosis and chooses Approve & Deploy, Edit Commands, or Reject.

Features

- Select a network troubleshooting case from the bundled dataset.
- Display the symptom, topology information, and show-command output.
- Run a deterministic rule-based diagnosis first.
- Use an AI model when no deterministic rule matches.
- Show:
  - Root cause
  - OSI layer
  - Confidence
  - Evidence
  - Next command to confirm
  - Proposed fix steps
- Record operator decisions in audit_log.csv.
- View case statistics and the human review/audit log in the Summary tab.

How It Works

1. The user selects a case from cases.csv.

2. checker.py checks the case using deterministic pattern matching and simple network calculations.

3. If a known fault is found, the rule engine returns a high-confidence diagnosis.

4. If no deterministic rule matches, engine.py sends the case to the configured AI model.

5. The AI response is converted into the same structured diagnosis format.

6. The Streamlit dashboard displays the result.

7. The operator reviews the evidence and selects Approve & Deploy, Edit Commands, or Reject.

8. The decision is saved to audit_log.csv.

The rule checker is designed not to guess: it only reports faults when it finds supporting evidence. If the AI key is unavailable, the application reports that honestly instead of inventing a diagnosis.

Project Structure

```
├── app.py
├── checker.py
├── engine.py
├── cases.csv
├── system_config.json
├── requirements.txt
├── diagnose_prompt.md
└── audit_log.csv          # created after decisions are reviewed
```

File Description

app.py

Main Streamlit dashboard. It loads cases, runs diagnoses, displays results, provides human review buttons, and shows summary/audit information.

checker.py

Deterministic rule engine. It checks show-command output and topology information for known network problems such as interface shutdowns, DHCP exhaustion, DNS lookup configuration, OSPF mismatches, ACL problems, NAT issues, VLAN/trunk problems, routing issues, and other lab faults.

engine.py

Diagnosis orchestrator. It runs the deterministic checker first. If no rule matches, it calls the AI diagnosis path and returns a consistent structured result to the dashboard.

cases.csv

Contains the bundled network troubleshooting scenarios used by the application.

system_config.json

Contains application configuration such as the application name, AI model, token limit, confidence setting, and paths to the cases, audit log, and prompt template.

requirements.txt

Lists the Python dependencies required to run the project.

diagnose_prompt.md

Prompt/template used when a case is handed to the AI model.

audit_log.csv

Created automatically when an operator reviews a diagnosis. It stores the timestamp, case ID, diagnosis source, rule, root cause, confidence, decision, and operator note.

Requirements

- Python 3.9+ recommended
- Streamlit
- Pandas
- Anthropic Python SDK

Install dependencies with:

pip install -r requirements.txt

AI Configuration

The deterministic rule engine can run without an AI API key.

For AI-assisted diagnosis, configure the ANTHROPIC_API_KEY environment variable.

Windows PowerShell

$env:ANTHROPIC_API_KEY="your_api_key_here"

Windows Command Prompt

set ANTHROPIC_API_KEY=your_api_key_here

macOS/Linux

export ANTHROPIC_API_KEY=your_api_key_here

If the API key is not configured, rule-based cases still work. Cases that require AI assistance are reported as unavailable rather than receiving a made-up answer.

Run the Application

From the project folder:

streamlit run app.py

Streamlit will provide a local URL, normally:

http://localhost:8501

Open that URL in a browser.

Using the Dashboard

Diagnose a Case

1. Open the Diagnose a case tab.
2. Select a case.
3. Review the symptom, topology note, and show-command output.
4. Click Run diagnosis.
5. Review the diagnosis and evidence.
6. Choose one of:

o Approve & Deploy

o Edit Commands

o Reject

7. Optionally add an operator note.

Summary & Audit Log

The Summary & audit log tab displays:

- Cases grouped by issue type.
- Cases grouped by OSI layer.
- Human review decisions.
- Diagnosis source split.

Safety / Human Review

NetSage AI is designed as a troubleshooting assistant, not an autonomous network deployment system. The diagnosis and proposed commands are presented for human review. The application records the operator's decision instead of automatically applying configuration changes.

Example

If a case reports that an interface is administratively down, the deterministic rule engine can identify the issue directly from the show-command output.

Example diagnosis:

Root cause: Interface is administratively down

Next command: show ip interface brief

Suggested fix:

configure terminal

interface <interface>

no shutdown

The operator can inspect the evidence and then record the appropriate decision.

Notes

- Deterministic rules are checked before the AI path.
- The application uses the same structured diagnosis format for both rule-based and AI-generated results.
- AI failures and missing API configuration are surfaced to the operator with low-confidence status.
- Review decisions are persisted in audit_log.csv.